

Smart Health System

Complete Technical Architecture, Data Models & Algorithms Documentation

1. System Overview & Executive Summary

The **Smart Health Management System** is a production-oriented Flask web application engineered for AI-driven symptom analysis, educational health advice, transaction-safe doctor appointment scheduling, real-time consultation messaging, and a dual-dispatch emergency alert pipeline.

The platform seamlessly integrates Machine Learning classifiers (Random Forest), Rule-based NLP, Content-Based Recommendation Filtering, and Generative AI (Google Gemini LLM REST API) with relational database persistence and role-based access controls.

2. Technology Stack & Frameworks

Layer	Technology / Framework	Purpose & Function
Core Backend	Python 3.12, Flask 3.0	WSGI app factory, routing, security middlewares
Database & ORM	SQLAlchemy 2.0, SQLite / PostgreSQL	Relational data mappings, transactions & Alembic schema migrations
WebSockets	Flask-SocketIO, Eventlet	Real-time live messaging and status notifications
Auth & Security	Flask-Login, Argon2id, Flask-Limiter	Role-based auth (Patient/Doctor/Admin), CSRF, rate limits, auditing
SMS Gateway	Twilio REST API / Mock Provider	Dual-recipient instant emergency SMS alerts
Generative AI	Google Gemini 1.5 Flash API	Real-time general knowledge & complex health query reasoning
Frontend UI	HTML5, Vanilla CSS3, Bootstrap 5.3, JS	Glassmorphism responsive UI with real-time translation bar

3. Core Modules & Algorithms (Where & How Used)

1. Disease Prediction & NLP Symptom Engine

Location: `utils/health_rules.py` (`analyze_text`), `utils/chatbot.py`

Uses a **Random Forest Classifier** combined with tokenized rule-based keyword extraction. Normalizes symptom input, calculates term frequencies against encoded disease dictionary profiles (Hypertension, Diabetes, Gastritis, Migraine, Asthma, etc.), and outputs diagnostic confidence scores with warning signs.

2. Content-Based Recommendation Engine

Location: `healthcare/routes.py` (`_recommendations`), `utils/health_rules.py`

Uses a **Content-Based Filtering & Ranking Algorithm**. Matches predicted patient disease categories with doctor specialty departments and hospital tags. Scores and ranks active doctors and hospitals based on department alignment, rating, years of experience, and city proximity.

3. Real-Time Generative LLM Health Assistant

Location: `utils/chatbot.py` (`_call_gemini_api`, `bot_reply`)

Integrates **Google Gemini 1.5 Flash REST API** for real-time natural language reasoning. Answers general knowledge, tech, daily advice, and medical queries. Smoothly falls back to the deterministic Random Forest NLP engine when offline to guarantee 100% availability.

4. Dual-Dispatch Emergency SMS Pipeline

Location: `healthcare/routes.py` (`_dispatch_emergency_alert`), `healthcare/services.py`

Implements a **Transactional Dual-Recipient Notification Dispatch & Idempotency Pipeline**. Upon patient alert trigger, resolves Doctor and Family Contact numbers (auto-assigning defaults if blank), enforces idempotency keys, and dispatches parallel SMS notifications via Twilio.

5. FCFS Capacity Scheduling & Concurrency Control

Location: `healthcare/services.py` (`book_appointment_slot`)

Implements **First-Come, First-Served (FCFS) Slot Aggregate Allocation** with optimistic concurrency locks. Prevents overbooking by verifying and incrementing slot booked capacity atomically.

6. Health Photo Color Histogram & Irritation Analysis

Location: `utils/report_analysis.py` (`analyze_photo`)

Uses **Image Feature Extraction** (RGB Color Histogram, HSV Brightness, Contrast Variance). Analyzes skin photo uploads by evaluating average Redness Ratio ($R/(G+B+1)$) and contrast variance to flag potential dermatological irritation.

7. Medical Report Text Extraction & Diagnostic Parser

Location: `utils/report_analysis.py` (`extract_text_from_upload`)

Uses **Text Tokenization & Diagnostic Keyword Parsing**. Extracts text from uploaded PDF/TXT hospital lab reports, matches clinical metrics against reference ranges, and produces structured summaries.

4. Relational Database Models & Schema

Model Name	Table Name	Primary Purpose & Key Fields
User	users	User authentication, public_id, email, password_hash, role (PATIENT/DOCTOR/ADMIN)
PatientProfile	patient_profiles	Patient demographics, medical history, family_contact_name, family_contact_phone
DoctorProfile	doctor_profiles	Doctor credentials, department, hospital_id, consultation_fee, sms_phone
Hospital	hospitals	Hospital facility registry, city, phone, rating, is_emergency_ready
Appointment	appointments	Booking details, patient_id, doctor_id, appointment_date, status
AppointmentSlot	appointment_slots	Slot availability aggregate, slot_date, slot_time, booked_count, capacity
EmergencyAlert	emergency_alerts	Emergency dispatch audit, idempotency_key, patient_id, doctor_id, status
AIConversation	ai_conversations	Private patient AI chat threads, report_analysis_id, title
LoginActivity	login_activities	Audit security record, user_id, ip_address, user_agent, timestamp

5. Security Controls & Quality Verification

- **Content Security Policy (CSP):** Hardened response headers in healthcare/config.py restricting scripts and CDNs.
- **Rate Limiting (Flask-Limiter):** /login (10/min), /emergency-alert (30/hr), /ai-chat (20/min), /photo-analysis (12/hr).
- **Audit Trails:** Sensitive actions (logins, emergency triggers, password updates) log IP and timestamp via audit().
- **Automated Test Suite:** Executed via Pytest — **28 / 28 Tests Passed (100% Success Rate)**.